TerraSentinel-Edge

1. Descriere

Proiectul este un sistem care procesează imagini satelitare Sentinel-2 și găsește automat zonele afectate de secetă, rulând direct pe o placă NVIDIA Jetson. Din benzile vizibile, NIR și SWIR calculez indici de vegetație și de stres hidric (NDVI, NDMI, NDWI, MSI), iar un model de segmentare semantică produce harta zonelor aride.

CPU-ul ARM al plăcii citește imaginile de pe disc, aduce benzile la aceeași rezoluție și coordonează aplicația. GPU-ul preia calculele grele: kernelurile CUDA calculează indicii direct în memoria partajată, iar rețeaua neuronală rulează optimizat prin TensorRT.

La final compar varianta accelerată cu cea rulată doar pe procesor: cât de precisă e segmentarea, ce abateri numerice apar la cuantizare (FP16 și INT8) și cât se reduce timpul total de execuție.

2. Tehnologii

- **Python:** citirea fișierelor satelitare (rasterio), aducerea benzilor la 10 m, calculele de referință pe CPU (NumPy), antrenarea modelului (PyTorch) și vizualizarea hărților.
- **CUDA C++ și TensorRT:** kernelurile paralele pentru indici, cu memorie unificată, și optimizarea rețelei prin fuziunea straturilor și cuantizare la precizie redusă.
- **NVIDIA Jetson:** platforma țintă, unde ARM-ul coordonează datele și operațiile de I/O, iar GPU-ul accelerează preprocesarea și inferența la nivel de pixel.
- **Git și CMake:** versionarea codului și compilarea bibliotecii partajate pe ARM.

3. Roadmap

Săptămâna	Task	Descriere	Fișiere estimate
6	**Documentare și pregătirea datelor**	• Studiez benzile Sentinel-2 care contează pentru vegetație și umiditate: B4 (roșu) și B8 (NIR) la 10 m, B8A, B11 și B12 (SWIR) la 20 m. • Aleg indicii spectrali: NDVI, NDMI, NDWI și MSI. • Descarc 5–10 scene din Copernicus Browser (zone agricole din România și Spania, vară secetoasă vs. normală). • Stabilesc de unde iau etichetele: măști din date publice (ex. Copernicus Drought Observatory) sau etichetare manuală pe patch-uri mici. • Configurez repository-ul Git, mediul Python și verific JetPack/CUDA pe placă.	`README.md` — descrierea proiectului și pașii de rulare `docs/Roadmap_TerraSentinel.pdf` `requirements.txt` — dependențele Python
8	**Baseline software (doar CPU) și antrenarea rețelei**	• Încarc benzile cu rasterio, aduc B8A, B11 și B12 de la 20 m la 10 m	`src/sw/band_loader.py` — încarcă benzile, le aliniază la 10 m și normalizează

Săptămâna	Task	Descriere	Fișiere estimate
		(interpolare bilineară) și normalizez valorile. • Calculez indicii în NumPy și într-o variantă cu bucle simple, ca să am și o referință lentă. • Tăiez scenele în patch-uri de 256×256. Fac split train/val/test pe scene întregi, nu pe patch-uri, ca să nu am scurgeri de date. • Antrenez un U-Net mic, îl evaluez cu IoU și F1 și îl export în ONNX. • Măsor timpii pe CPU-ul ARM al plăcii, ca baseline.	**src/sw/cpu_indices.py** — indicii pe CPU și măștile de referință **src/sw/train_segmentation.py** — definește U-Net-ul, antrenează și exportă ONNX **data/README.md** — sursa scenelor și a etichetelor
10	**Kernelurile CUDA și optimizarea TensorRT**	• Scriu kernelurile CUDA (un thread per pixel) pentru NDVI, NDMI, NDWI și MSI. • Folosesc memorie unificată (cudaMallocManaged) și o compar cu varianta clasică cu cudaMemcpy, ca să văd câștigul real. • Testez modulele izolat pe matrice sintetice cu rezultat cunoscut, inclusiv cazul împărțirii la zero. • Convertesc ONNX în motor TensorRT: întâi FP16, apoi INT8 cu un set de calibrare.	**src/hw/spectral_kernels.cu** — kernelurile pentru indici **src/hw/trt_engine.cpp** — construiește motorul TensorRT (FP16/INT8) **src/hw/hw_unit_test.cu** — teste pe date sintetice **src/sw/make_calib_set.py** — setul de calibrare pentru INT8
12	**Integrarea Python – C++ pe Jetson**	• Compilez codul CUDA și TensorRT ca bibliotecă partajată pe ARM, cu CMake. • O apelez din Python prin ctypes (sau pybind11), transmițând pointerii către benzile încărcate. • Leg tot pipeline-ul: scena brută, tiling, indici pe GPU, inferență TensorRT, harta finală de secetă. • Verific că rezultatele de pe CPU și GPU coincid, cu o toleranță numerică stabilită dinainte (ex. eroare maximă 1e-5 la indici).	**src/sw/jetson_pipeline.py** — apelează biblioteca și returnează harta **CMakeLists.txt** — reguli de build pentru codul CUDA
14	**Profilare, validare și documentație**	• Rulez sistemul pe 2–3 scene noi, nefolosite la antrenare. • Compar FP32, FP16 și INT8 din punct de vedere al IoU, F1, erorii numerice și al timpului (preprocesare, inferență, total).	**src/sw/benchmark_eval.py** — compară CPU vs. GPU și calculează speedup-ul **src/sw/visualize_maps.py** — hărți și grafice **docs/Documentatie_Proiect.docx**

Săptămâna	Task	Descriere	Fișiere estimate
		• Calculez speedup-ul și măsor memoria și consumul de putere cu tegrastats. • Generez hărțile vizuale și graficele comparative, apoi scriu documentația finală.	

4. Riscuri și plan B

- **Etichete insuficiente sau nesigure:** dacă nu găsesc măști bune, folosesc patch-uri etichetate manual (mai puține, dar corecte).
- **INT8 pierde prea multă precizie:** rămân pe FP16 și documentez comparația dintre cele trei precizii.
- **Memorie insuficientă pe Jetson:** scenele Sentinel-2 sunt mari (10980×10980 pixeli), așa că reduc dimensiunea tile-ului sau procesez pe bucăți.
- **Integrarea Python – C++ merge greu:** trec de la pybind11 la ctypes (sau invers) și păstrez testele izolate din săptămâna 10.

5. Rezultat final

Un sistem funcțional pe NVIDIA Jetson, care preia o captură multispectrală Sentinel-2, extrage automat indicii de vegetație și umiditate și segmentează zonele afectate de secetă.

Atât calculul indicilor, cât și inferența rețelei există în două variante: una de referință, rulată doar pe CPU-ul ARM, și una accelerată pe GPU cu CUDA C++ și TensorRT. Le compar pe baza acurateței hărților (IoU, F1), a stabilității numerice la precizie redusă și a timpului total de procesare, ca să arăt ce câștig aduce GPU-ul embedded.